BOSS 6 – GENERALPROBE · Welt 6 · DS 37 · 90 min · Abiturbedingungen

Probeklausur: Die Paketstation

Erreichbar: 35 BE · Generalprobe auf Papier, nicht benotet · **Zusätzlich im Lab: Online-Boss „Generalprobe online“**
Niveau (KC 2017): Grund- und Leistungskurs – f) bis h) (Rekursion) nur Leistungskurs (eA)

Der Endgegner wartet: In der nächsten Woche schreibst du die Klausur im Abitur-Format. Heute ist die Generalprobe – mit allem, was dazugehört: Zeitdruck, Material lesen, handschriftlich programmieren. Danach weißt du genau, wo deine Punkte liegen bleiben. Viel Erfolg!

Hinweise zur Bearbeitung (wie im Abitur)

- Bearbeitungszeit **90 Minuten**. Hilfsmittel: nur die **Ergänzenden Hinweise** (Auszug im Material) – keine Werkzeugkarten, kein Gerät.
- Bearbeite die Aufgaben **auf eigenem Papier** (Klausurbogen). Schreibe auf jedes Blatt deinen Namen und kennzeichne jede Lösung mit dem Buchstaben der Teilaufgabe. **Ausnahme:** Die Tabellen in den Abbildungen 4 und 7 füllst du direkt im Material aus und gibst das Material mit ab.
- Implementierungen schreibst du in **Java** mit den Klassen Stack, Queue und DynArray der Bibliothek „Niedersachsen“ (ohne Generics, mit Casts). Struktogramme nach den Ergänzenden Hinweisen (§5).
- Laufzeitfehler musst du durch **explizite Abfragen** ausschließen.
- Achte auf die **Operatoren** (nennen, erläutern, begründen, implementieren, entwickeln, dokumentieren, analysieren …).

Strategie für die Generalprobe

1. Überfliegen (3 min): Worum geht es, welche Klassen, welche Abbildungen? **2. Material markieren:** Klassenkarten, Signaturen, Beispielwerte. **3. BE-Plan:** ca. **2,5 Minuten je BE** – trage dir die Zielzeiten in die Tabelle ein. **4. Reserve:** 5 bis 10 Minuten zum Prüfen (Abi-Check: Leer? Index? Cast? equals? return auf jedem Pfad?).

Bewertung

Teilaufgabe	a)	b)	c)	d)	e)	f)	g)	h)	Summe
max. BE	4	7	6	4	2	5	2	5	35
Zielzeit (min)									90
Selbstkorrektur									
Peer-Korrektur									

Name: _____ Team: _____ Korrigiert von: _____

Aufgabe: Die Paketstation

Ein Paketdienst betreibt automatische Paketstationen. Jede Station besteht aus Fächern der Größen S, M und L. Ein Fach ist entweder frei oder enthält genau ein Paket. Pakete, die die Zustellerin in die Station einlegen soll, werden als Zustellaufträge in der Reihenfolge ihres Eintreffens verwaltet. Findet sich für ein Paket kein passendes freies Fach, wird es auf einen Stapel von Rücksendungen gelegt, den am Abend ein Kurierfahrzeug abholt.

Die Modellierung ist in Abbildung 1 im Material als Klassendiagramm dargestellt. Abbildung 2 zeigt den Zustand der Station Wiesenau.

a) Erläutern Sie am Beispiel der Klasse Paketstation die Funktion der beiden Konstruktoren.

Erläutern Sie außerdem, wie ein Objekt der Klasse Paketstation an die Sendungsnummer des Pakets in einem Fach gelangt, obwohl das Attribut inhalt der Klasse Fach mit „–“ gekennzeichnet ist.

[4 BE]

b) Die Operation **sucheFach(sendungsnr: Zeichenkette): Ganzzahl** der Klasse Paketstation liefert die Fachnummer des Fachs, in dem das Paket mit der übergebenen Sendungsnummer liegt. Liegt das Paket in keinem Fach der Station, wird `-1` zurückgegeben.

Beispiel: Für die Station aus Abbildung 2 liefert `sucheFach("P-2208")` den Wert 103.

Implementieren Sie die Operation sucheFach in einer im Unterricht verwendeten Programmiersprache. Laufzeitfehler sind durch geeignete Abfragen auszuschließen.

[7 BE]

c) Die Operation **anzahlAuftraege(groesse: Zeichen): Ganzzahl** der Klasse Paketstation liefert die Anzahl der Zustellaufträge in der Schlange auftraege, deren Paket die übergebene Größe hat. Nach Ausführung der Operation muss die Schlange auftraege unverändert sein (dieselben Pakete in derselben Reihenfolge).

Beispiel: Für die Station aus Abbildung 2 liefert `anzahlAuftraege('M')` den Wert 3.

Entwickeln Sie ein Struktogramm für die Operation anzahlAuftraege.

[6 BE]

d) Mit der Operation **zustellen** (Abbildung 3) legt die Zustellerin die Pakete der Zustellaufträge in passende Fächer.

Dokumentieren Sie die Verarbeitung der Zustellaufträge aus Abbildung 2 durch die Operation zustellen, indem Sie die Tracetabelle in Abbildung 4 vervollständigen. Tragen Sie für jeden Durchlauf der äußeren Schleife eine Zeile mit den Werten am Ende des Durchlaufs ein (nach der Verzweigung „nicht gefunden“, dem letzten Schritt im Rumpf der äußeren Schleife) und geben Sie die Rückgabe an. Zu Beginn ist der Stapel ruecksendungen leer.

[4 BE]

e) Ein Paket der Größe S passt auch in ein Fach der Größe M oder L, ein Paket der Größe M auch in ein Fach der Größe L.

Erläutern Sie, wie die Operation zustellen verändert werden muss, damit ein Paket nur dann in den Stapel ruecksendungen gelegt wird, wenn kein freies Fach mindestens seiner Größe vorhanden ist. Dabei soll jeweils ein möglichst kleines freies Fach verwendet werden.

[2 BE]

f) Am Abend holt ein Kurierfahrzeug die Rücksendungen ab. In seinem Laderaum sind noch platz Volumeneinheiten frei; ein Paket der Größe S belegt 1, M belegt 2 und L belegt 3 Volumeneinheiten (Operation getVolumen). Die Operation **sammle** (Abbildung 5) nimmt Pakete vom Stapel, solange sie in den Laderaum passen, und liefert ihre Anzahl.

Dokumentieren Sie den Ablauf des Aufrufs `sammle(s, 8)` für den Stapel s aus Abbildung 6 in der Tabelle in Abbildung 7.

Erläutern Sie das Prinzip der Rekursion anhand der Operation sammle.

[5 BE]

g) Begründen Sie, dass die Rekursionstiefe beim Aufruf `sammle(s, platz)` mit platz ≥ 0 höchstens den Wert platz erreicht – unabhängig davon, wie viele Pakete im Stapel s liegen. **[2 BE]**

h) Die Methode **freieFaecher** (Abbildung 8) soll rekursiv die Anzahl der freien Fächer der Größe groesse ab dem Index i bestimmen; der Erstaufruf erfolgt mit i = 0. Für die Station aus Abbildung 2 liefert `freieFaecher('M', 0)` jedoch den Wert 0 statt 2 und `freieFaecher('L', 0)` den Wert 0 statt 1.

Analysieren Sie die Methode. Erläutern Sie die beiden Fehler und korrigieren Sie die Methode. Untersuchen Sie außerdem den Aufruf `freieFaecher('M', 0)` für eine Station ohne Fächer. **[5 BE]**

Summe: 35 BE

Material

zu allen Aufgabenteilen

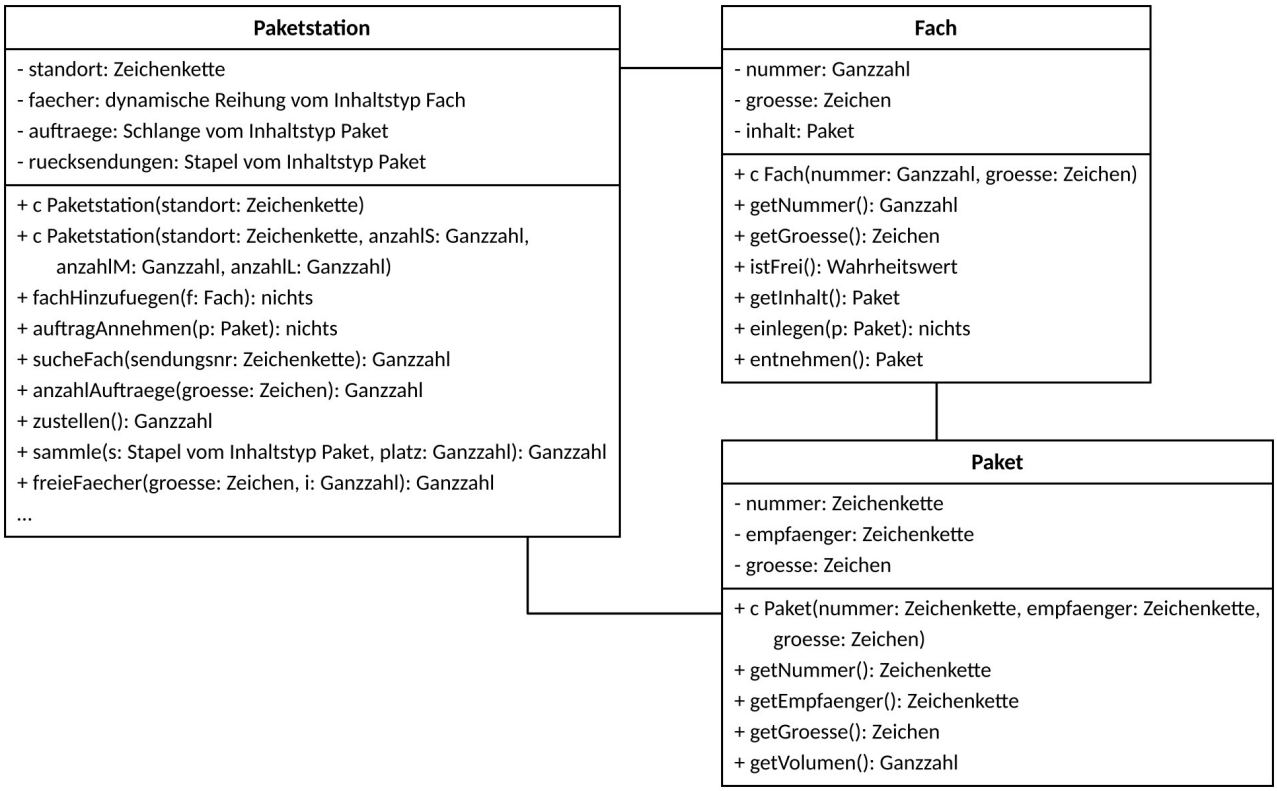


Abbildung 1: Klassendiagramm der Paketstation. Ist ein Fach frei, hat inhalt den Wert null (getInhalt() liefert dann null). getVolumen() liefert für die Größe S den Wert 1, für M den Wert 2 und für L den Wert 3.

Die Station Wiesenau wurde mit new `Paketstation("Wiesenau", 2, 3, 2)` erzeugt; die Fächer werden dabei ab 101 fortlaufend nummeriert. Inzwischen liegen in drei Fächern Pakete, und sechs Zustellaufträge warten:

faecher (dynamische Reihung vom Inhaltstyp Fach)

Index	0	1	2	3	4	5	6
Fachnummer	101	102	103	104	105	106	107
Größe	S	S	M	M	M	L	L
inhalt	P-7310 (S) Yilmaz	frei	P-2208 (M) Brandt	frei	frei	P-5512 (L) Okafor	frei

auftraege (Schlange vom Inhaltstyp Paket)

P-4127 (M)	P-9054 (L)	P-1863 (S)	P-6620 (M)	P-3089 (M)	P-8845 (L)

vorn (head, dequeue) *hinten (enqueue)*

Abbildung 2: Zustand der Station Wiesenau – Fächer (Index, Fachnummer, Größe, Inhalt; freie Fächer grün) und Zustellaufträge (vorn steht der zuerst eingetroffene Auftrag)

zu den Aufgabenteilen d) und e)

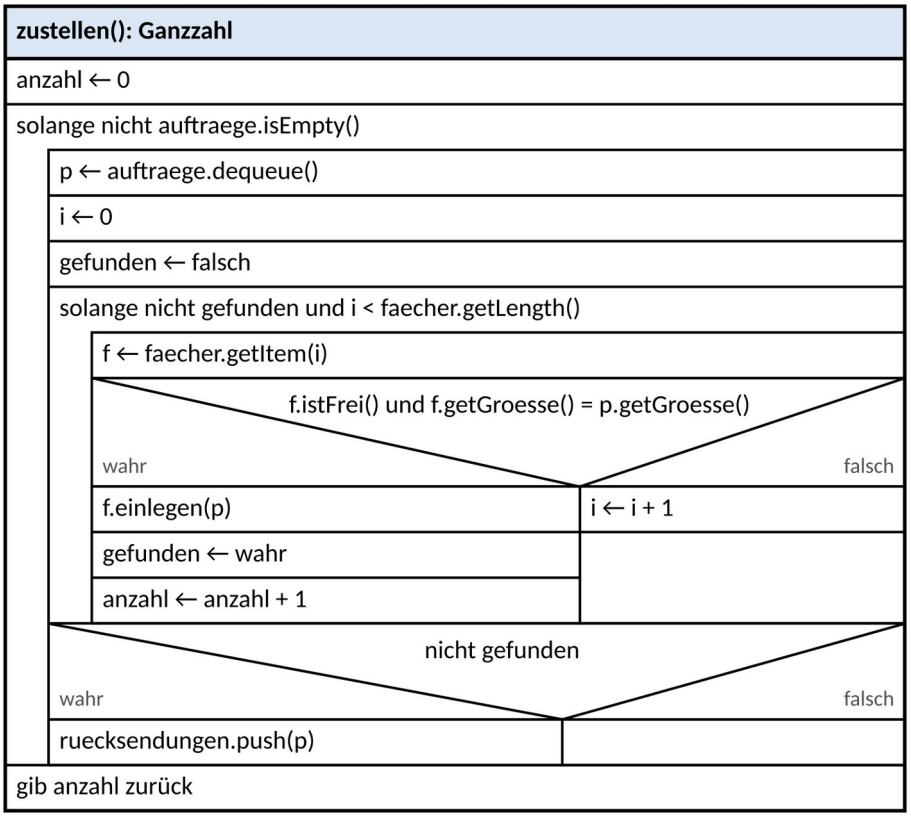


Abbildung 3: Struktogramm der Operation zustellen der Klasse Paketstation

zu Aufgabenteil d)

Durchlauf	p	i	gefunden	eingelegt in Fach	ruecksendungen (oben zuerst)	anzahl
vor der Schleife	–	–	–	–	–	0
1						
2						
3						
4						
5						
6						

Rückgabe: _____

Abbildung 4: Tracetabelle zur Vervollständigung (Zeile „vor der Schleife“ nach anzahl ← 0, dann eine Zeile je Durchlauf der äußeren Schleife, Werte am Ende des Durchlaufs; p mit Sendungsnummer und Größe, z. B. P-4127 (M))

zu den Aufgabenteilen f) und g)

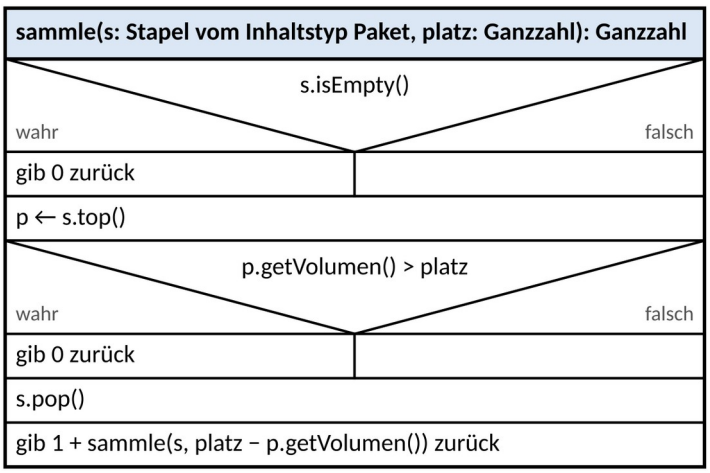


Abbildung 5: Struktogramm der Operation sammle

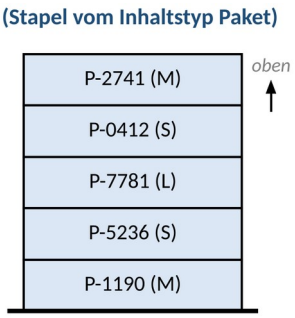


Abbildung 6: Stapel s (Rücksendungen)

Rekursions-tiefe *)	Stapel s beim Aufruf (oben zuerst)	platz	p	p.getVolumen()	Rückgabe
0 (Erstaufruf)	P-2741 (M), P-0412 (S), P-7781 (L), P-5236 (S), P-1190 (M)	8			

Rückgabe des Erstaufrufs: _____ Inhalt von s danach (oben zuerst): _____

**) Bei jedem rekursiven Aufruf erhöht sich die Rekursionstiefe um 1. Der Erstaufruf hat die Rekursionstiefe 0.*

Abbildung 7: Tabelle zur Dokumentation des Aufrufs sammle(s, 8) zur Vervollständigung

zu Aufgabenteil h)

```
 1  public int freieFaecher(char groesse, int i) {
 2      if (i == faecher.getLength() - 1) {
 3          return 0;
 4      }
 5      Fach f = (Fach) faecher.getItem(i);
 6      int anzahl = 0;
 7      if (f.istFrei() && f.getGroesse() == groesse) {
 8          anzahl = 1;
 9      }
10      freieFaecher(groesse, i + 1);
11      return anzahl;
12  }
```

Abbildung 8: Fehlerhafte Implementierung der Methode freieFaecher der Klasse Paketstation

Hilfsmittel: Auszug aus den Ergänzenden Hinweisen (§4 Datenstrukturen)

Datenstruktur	Operationen	Beschreibung
Stapel (Stack)	c Stack() · isEmpty(): Wahrheitswert push(x: Inhaltstyp): nichts pop(): Inhaltstyp top(): Inhaltstyp	push legt x oben auf den Stapel. pop entfernt das oberste Element und liefert es, top liefert es, ohne es zu entfernen – beide nur bei nicht leerem Stapel.
Schlange (Queue)	c Queue() · isEmpty(): Wahrheitswert enqueue(x: Inhaltstyp): nichts dequeue(): Inhaltstyp head(): Inhaltstyp	enqueue stellt x hinten an. dequeue entfernt das vorderste Element und liefert es, head liefert es, ohne es zu entfernen – beide nur bei nicht leerer Schlange.
dynamische Reihung (DynArray)	c DynArray() · isEmpty(): Wahrheitswert getLength(): Ganzzahl getItem(i: Ganzzahl): Inhaltstyp append(x: Inhaltstyp): nichts insertAt(i: Ganzzahl, x: Inhaltstyp): nichts setItem(i: Ganzzahl, x: Inhaltstyp): nichts delete(i: Ganzzahl): nichts	Gültige Indizes: 0 bis getLength() − 1. append hängt hinten an. insertAt fügt an Position i ein, die folgenden Elemente rücken nach hinten (i = getLength() hängt an). setItem ersetzt das Element an Position i. delete entfernt es, die folgenden Elemente rücken nach vorn.

Im Abitur liegen die vollständigen Ergänzenden Hinweise aus (u. a. Struktogramm-Notation §5, Klassendiagramme §6). In Java (Online-IDE, Bibliothek „Niedersachsen“) heißen die Klassen Stack, Queue und DynArray; sie sind nicht generisch, Inhalte werden beim Entnehmen gecastet, z. B. `Fach f = (Fach) faecher.getItem(i);`.